Online AuxIVA-ISS and AuxIVA-IP

Implementation, verification, and a runtime study against the batch algorithms

Abstract

This report documents a from-scratch implementation of online (frame-recursive) independent vector analysis using two update rules, iterative source steering (ISS) and iterative projection (IP), together with batch reference implementations of both. It covers the mathematics of each algorithm, a detailed walk through the code, the verification strategy used to establish correctness, and the measured results.

The central finding is that the speed advantage ISS enjoys offline does not transfer to the online setting. Offline it grows with channel count, from 0.83× at two channels to 2.68× at sixteen. Online it decays over the same range, from 1.34× to 0.85×. The reason is structural rather than an artefact of implementation, and is explained in Section 8.

Six defects were found and corrected along the way, several of which changed reported numbers materially. These are documented in Section 6 rather than quietly fixed, because each is easy to reintroduce.

Contents

1 Problem setting and notation

We separate K sound sources recorded by M microphones. Convolutive mixing in the time domain becomes approximately multiplicative in each narrow frequency band, so in the short-time Fourier transform (STFT) domain

$$\boldsymbol{x}_{fn} = \boldsymbol{A}_f \boldsymbol{s}_{fn}, \tag{1}$$

where $f = 1, \dots, F$ indexes frequency, $n = 1, \dots, N$ indexes time frame, $\boldsymbol{A}_f$ is the mixing matrix, and $\boldsymbol{s}_{fn}$ collects the source signals. This approximation holds when the transform is long relative to the room impulse response, a point that returns in Section 7.3.

Throughout we assume the determined case $M = K$, so $\boldsymbol{A}_f$ is square and invertible. The goal is a demixing matrix $\boldsymbol{W}_f = [\boldsymbol{w}_{1f} \cdots \boldsymbol{w}_{Mf}]^{\mathsf{H}}$ such that

$$\boldsymbol{y}_{fn} = \boldsymbol{W}_f \boldsymbol{x}_{fn} \tag{2}$$

recovers the sources. Note carefully that the *rows* of $\boldsymbol{W}_f$ are the conjugate transposes $\boldsymbol{w}_{kf}^{\mathsf{H}}$; this convention causes a specific class of implementation bug, discussed in Section 4.1.

1.1 The cost function

Under the assumptions that sources are independent and follow a spherical super-Gaussian distribution, the negative log-likelihood is

$$\mathcal{L} = \sum_{k=1}^{M} G\Big(\sqrt{\textstyle\sum_f |\boldsymbol{w}_{kf}^{\mathsf{H}} \boldsymbol{x}_{fn}|^2}\Big) - 2 \sum_f \log |\det(\boldsymbol{W}_f)|. \tag{3}$$

Direct minimisation is awkward, so auxiliary function (majorisation-minimisation) optimisation is used. The cost is majorised by

$$\mathcal{L} \le \mathcal{L}_2 = \sum_f \sum_k \boldsymbol{w}_{kf}^{\mathsf{H}} \boldsymbol{V}_{kf} \boldsymbol{w}_{kf} - 2 \sum_f \log |\det(\boldsymbol{W}_f)|, \tag{4}$$

with the auxiliary variables

$$r_{kn} = \sqrt{\sum_f |\boldsymbol{w}_{kf}^{\mathsf{H}} \boldsymbol{x}_{fn}|^2}, \qquad \boldsymbol{V}_{kf} = \frac{1}{N} \sum_n \varphi(r_{kn})\, \boldsymbol{x}_{fn} \boldsymbol{x}_{fn}^{\mathsf{H}}. \tag{5}$$

Here $\varphi(r) = G'(r)/(2r)$ is a positive weight that is small when a source is active and large when it is not. Two source models are supported:

$$\varphi_{\text{Laplace}}(r) = \frac{1}{2r}, \qquad \varphi_{\text{Gauss}}(r) = \frac{F}{r^2}. \tag{6}$$

Both algorithms below minimise the same $\mathcal{L}_2$. They differ only in *how* they descend it, which is why they reach equivalent separation quality and differ only in cost. Every experiment in this report confirms that equivalence, typically agreeing to within 0.03 dB.

2 The batch algorithms

2.1 AuxIVA-IP: iterative projection

IP minimises $\mathcal{L}_2$ with respect to one row of $\boldsymbol{W}_f$ at a time, holding the others fixed. Each row has a closed-form solution:

$$\boldsymbol{w}_{kf} \leftarrow (\boldsymbol{W}_f \boldsymbol{V}_{kf})^{-1} \boldsymbol{e}_k, \qquad \boldsymbol{w}_{kf} \leftarrow \frac{\boldsymbol{w}_{kf}}{\sqrt{\boldsymbol{w}_{kf}^{\mathsf{H}} \boldsymbol{V}_{kf} \boldsymbol{w}_{kf}}}. \tag{7}$$

The first step is a linear solve; the second fixes the scale of the demixing vector. Per source and per iteration this requires building an $M \times M$ covariance and solving against it, which is the source of both its cost and its potential instability when $\boldsymbol{V}_{kf}$ becomes ill-conditioned.

2.2 AuxIVA-ISS: iterative source steering

ISS instead applies a rank-1 update to the *whole* matrix,

$$\boldsymbol{W}_f \leftarrow \boldsymbol{W}_f - \boldsymbol{v}_{kf}\, \boldsymbol{w}_{kf}^{\mathsf{H}}, \tag{8}$$

repeated for $k = 1, \ldots, M$. Minimising $\mathcal{L}_2$ over $\boldsymbol{v}_{kf}$ gives

$$v_{mk} = \begin{cases} \dfrac{\boldsymbol{w}_m^{\mathsf{H}} \boldsymbol{V}_m \boldsymbol{w}_k}{\boldsymbol{w}_k^{\mathsf{H}} \boldsymbol{V}_m \boldsymbol{w}_k} & m \neq k, \\ 1 - \left(\boldsymbol{w}_k^{\mathsf{H}} \boldsymbol{V}_k \boldsymbol{w}_k\right)^{-1/2} & m = k. \end{cases} \tag{9}$$

No matrix inverse appears. Two properties matter for what follows.

First, the quantities needed are only

$$\boldsymbol{w}_m^{\mathsf{H}} \boldsymbol{V}_m \boldsymbol{w}_k = \sum_n \varphi(r_{mn})\, y_{mn} y_{kn}^*, \qquad \boldsymbol{w}_k^{\mathsf{H}} \boldsymbol{V}_m \boldsymbol{w}_k = \sum_n \varphi(r_{mn})\, |y_{kn}|^2, \tag{10}$$

which are computable *directly from the separated outputs* y, by summing over time. The covariance matrices never need to be formed. Each costs $O(N)$, and M of them are needed per source update, giving $O(MN)$ per update.

Second, the output can be updated without ever storing $\boldsymbol{W}_f$ at all:

$$\boldsymbol{y}_n \leftarrow (\boldsymbol{W} - \boldsymbol{v}_k \boldsymbol{w}_k^{\mathsf{H}}) \boldsymbol{x}_n = \boldsymbol{y}_n - \boldsymbol{v}_k y_{kn}. \tag{11}$$

Offline this permits an in-place implementation using $O(1)$ extra memory. Section 8.1 shows that this is precisely what the online setting cannot exploit.

Note the index on $\boldsymbol{V}_m$ in (9): the covariance belongs to the row being updated, m, not to the pivot k. This is easy to get backwards and was verified against both the reference implementation and a brute-force rewrite.

3 The online algorithms

3.1 From batch statistics to a recursion

The batch auxiliary variable (5) averages over all N frames. Online, only the current frame is available, and a single frame yields a rank-1 matrix that carries no useful spatial information on its own. The standard remedy is to replace the batch average with an exponentially forgotten recursion,

$$\boldsymbol{U}_{kft} \leftarrow \alpha\, \boldsymbol{U}_{kf(t-1)} + (1 - \alpha)\, \varphi(r_{kt})\, \boldsymbol{x}_{ft} \boldsymbol{x}_{ft}^{\mathsf{H}}, \tag{12}$$

Algorithm 1: Online AuxIVA-ISS

Input: mixture $\boldsymbol{x}_{ft}$, iterations N_{iter}, forgetting factor α, initial $\boldsymbol{W}_{f0}$, initial $\boldsymbol{U}_{kf0}$
Output: separated $\boldsymbol{y}_{ft}$

for $t = 1, \ldots, T$ **do**
  $\boldsymbol{W}_{ft} \leftarrow \boldsymbol{W}_{f(t-1)}$
  for iter $= 1, \ldots, N_{\text{iter}}$ **do**
    for $k = 1, \ldots, K$ **do**
      $r_{kt} \leftarrow \sqrt{\sum_f |\boldsymbol{w}_{kft}^{\mathsf{H}} \boldsymbol{x}_{ft}|^2}$
      $\boldsymbol{U}_{kft} \leftarrow \alpha \boldsymbol{U}_{kf(t-1)} + (1-\alpha)\varphi(r_{kt}) \boldsymbol{x}_{ft} \boldsymbol{x}_{ft}^{\mathsf{H}}$
      for $m = 1, \ldots, M$ **do**
        $v_{mkf} \leftarrow \dfrac{\boldsymbol{w}_{mft}^{\mathsf{H}} \boldsymbol{U}_{mft} \boldsymbol{w}_{kft}}{\boldsymbol{w}_{kft}^{\mathsf{H}} \boldsymbol{U}_{mft} \boldsymbol{w}_{kft}}$ if $m \neq k$
        $v_{kkf} \leftarrow 1 - (\boldsymbol{w}_{kft}^{\mathsf{H}} \boldsymbol{U}_{kft} \boldsymbol{w}_{kft})^{-1/2}$
      $\boldsymbol{W}_{ft} \leftarrow \boldsymbol{W}_{ft} - \boldsymbol{v}_{kf} \boldsymbol{w}_{kft}^{\mathsf{H}}$
  $\boldsymbol{y}_{ft} = \boldsymbol{W}_{ft} \boldsymbol{x}_{ft}$

with forgetting factor α, typically 0.96. The effective memory is about $1/(1-\alpha)$ frames, roughly 25 at $\alpha = 0.96$. The auxiliary variable is computed from the current frame alone,

$$r_{kt} = \sqrt{\sum_f |\boldsymbol{w}_{kft}^{\mathsf{H}} \boldsymbol{x}_{ft}|^2}. \tag{13}$$

Per incoming frame, a small number of sweeps N_{iter} is applied, typically one to three, rather than the dozens of full passes the batch method makes. Frame t's output depends only on frames $\leq t$, so the separation is causal.

3.2 Online AuxIVA-ISS

Algorithm 3.2 states the procedure. It is the algorithm of Nakashima and Ono, restricted to the case where every source is updated on every sweep; the "flexible" variant that updates only moving sources is not implemented here and is noted as a limitation in Section 11.

Two details deserve emphasis, both of which were sources of error.

The recursion is anchored to the previous frame. In (12) the right-hand side uses $\boldsymbol{U}_{kf(t-1)}$, the covariance as it stood at the *end of the previous frame*, not the running value from the current sweep. The purpose of the inner loop is to recompute r_{kt} as $\boldsymbol{W}$ improves; decaying by α again, or injecting $\boldsymbol{x}\boldsymbol{x}^{\mathsf{H}}$ a second time, is not intended. Getting this wrong is silent: the algorithm still separates, just worse. Section 6 quantifies the cost at about 1.5 dB of SDR.

Sources are swept sequentially. Within a sweep, source $k+1$ sees the demixing matrix already updated by source k. This is Gauss-Seidel rather than Jacobi, and it is deliberate: the pivot row must be frozen while all rows are updated relative to it, but the next pivot uses the fresh matrix.

3.3 Online AuxIVA-IP

The IP baseline is structured identically, sharing the recursion (12), the auxiliary variable, and the per-frame sweep. Only the update differs: where ISS applies (8), IP performs the solve and normalisation of (7). This was a deliberate design choice, so that a timing comparison measures the update rule rather than incidental differences in implementation style.

4 Implementation

4.1 Storage conventions, and a trap

The demixing matrix is stored so that `y = W @ x` holds directly, with no extra conjugation at use time. Consequently row k of the stored array *is* $\boldsymbol{w}_k^{\mathsf{H}}$, and the true steering vector is its conjugate:

$$\texttt{W[f,k,:]} = \boldsymbol{w}_k^{\mathsf{H}}, \qquad \text{hence} \qquad \boldsymbol{w}_k = \overline{\texttt{W[f,k,:]}}. \tag{14}$$

This matters for the Hermitian forms in (9). Writing a for the stored row, the correct evaluation is

$$\boldsymbol{w}_k^{\mathsf{H}} \boldsymbol{U} \boldsymbol{w}_k = a\, \boldsymbol{U}\, \bar{a}, \tag{15}$$

with $\boldsymbol{U}$ applied to the *conjugated* row and the plain row contracted on the left. The natural-looking alternative $\bar{a}\, \boldsymbol{U}\, a$ is a different quantity. Both are real when $\boldsymbol{U}$ is Hermitian, so the error produces plausible numbers rather than obvious nonsense. It was caught by a direct numerical check:

```
true w_k^H U w_k = -1.7374291699646307
E1 (naive, using a twice)     = -2.722010666857075   # wrong
E2 (a on left, conj(a) via U) = -1.7374291699646305  # correct
```

The corresponding code reads:

```
# U[m] @ conj(w_k) for every source m, shape (n_src, n_freq, n_chan)
Uk_wk = (U @ np.conj(w_k_frozen)[None, :, :, None])[..., 0]

denom = np.real(np.sum(w_k_frozen[None, :, :] * Uk_wk, axis=-1))
denom = np.maximum(denom, eps)

W_rows = np.transpose(W, (1, 0, 2))
numer = np.sum(W_rows * Uk_wk, axis=-1)
```

Note that `numer` contracts the stored row a_m against $\boldsymbol{U}_m \bar{a}_k$, matching $\boldsymbol{w}_m^{\mathsf{H}} \boldsymbol{U}_m \boldsymbol{w}_k$, while `denom` contracts the pivot row, matching $\boldsymbol{w}_k^{\mathsf{H}} \boldsymbol{U}_m \boldsymbol{w}_k$. The two use the same intermediate.

4.2 The core loop

The online ISS update, with commentary:

```
for t in range(n_frames):
    x_t = X[t]                                      # (n_freq, n_chan)
    xxH = x_t[:, :, None] * np.conj(x_t[:, None, :])

    U_prev = U.copy()           # snapshot: the recursion anchors here

    for _ in range(n_iter):
        for k in range(n_src):
            w_k = W[:, k, :]
            y_k = np.sum(w_k * x_t, axis=-1)
            r_k = np.sqrt(np.sum(np.abs(y_k) ** 2))
            phi_k = _phi(r_k, model, n_freq, eps)

            U[k] = alpha * U_prev[k] + (1 - alpha) * phi_k * xxH

            w_k_frozen = w_k.copy()                 # pivot must not move
            # ... compute v as above ...
            W -= v_t[:, :, None] * w_k_frozen[:, None, :]

    Y_out[t] = (W @ x_t[:, :, None])[..., 0]
```

Three points. The snapshot `U_prev` implements the anchoring discussed above. The pivot row is copied before the update because the rank-1 subtraction modifies every row of $\boldsymbol{W}$, including

the pivot itself, and all v_m must be computed against the pre-update pivot. Finally, the frequency axis is fully vectorised while the frame, sweep, and source loops remain in Python; with $F = 513$ bins the interpreter overhead is negligible, since only $N_{\text{iter}} \times K$ Python iterations occur per frame.

4.3 What is and is not vectorised

Axis	Treatment	Reason
Frames	Python loop	Sequential by definition; the point of the algorithm
Sweeps	Python loop	Each refines the previous
Sources	Python loop	Gauss-Seidel; source $k+1$ needs k's result
Frequency	Vectorised	Independent across bins
Channels	Batched BLAS	Small dense operations

The channel-axis operations are written as batched `matmul` rather than `einsum`. This is not cosmetic: `einsum` does not dispatch to BLAS, and since the IP baseline uses `np.linalg.solve` and `@` which do, an `einsum`-based ISS would have compared implementation styles rather than algorithms. Before this was corrected, ISS appeared *slower* than IP above eight channels, which was an artefact.

4.4 Projection back

Blind separation recovers each source only up to an unknown complex gain *per frequency bin*. Left unresolved the output is spectrally distorted: it sounds filtered, regardless of how well the sources were actually separated. Projection back fits, by least squares, one complex coefficient per bin and per source against a reference microphone:

$$c_{fk} = \frac{\sum_n x_{fn}^{(0)*} y_{fnk}}{\sum_n |y_{fnk}|^2}, \qquad y_{fnk} \leftarrow \bar{c}_{fk}\, y_{fnk}. \tag{16}$$

For $F = 513$ and $K = 2$ this is 1026 coefficients, not a single gain. It is a filter, not a volume control. Verified against a brute-force optimal scalar to 5×10^{-16}.

Its effect is large. With projection back disabled, SDR measured -17.65 dB while SIR *improved* to 28.29 dB: the separation was working, but each bin carried an arbitrary gain.

5 Verification

Correctness was established by three independent means rather than inspection.

Brute-force cross-check. Both algorithms were reimplemented a second time using fully explicit scalar loops, written directly from the pseudocode and sharing no code with the vectorised versions. Agreement across both source models and two iteration counts:

Model	N_{iter}	ISS $\max \lvert\text{vec} - \text{brute}\rvert$	IP $\max \lvert\text{vec} - \text{brute}\rvert$
Laplace	1	3.6×10^{-14}	2.5×10^{-14}
Laplace	3	2.1×10^{-14}	2.6×10^{-14}
Gauss	1	9.2×10^{-14}	1.4×10^{-13}
Gauss	3	1.0×10^{-11}	8.3×10^{-14}

Method	Amari distance
Unseparated input	0.5318
Batch ISS	0.0023
Batch IP	0.0023
Online ISS	0.0085
Online IP	0.0085

Functional test with known ground truth. On a synthetic mixture whose sources follow the IVA dependency model and whose mixing matrix is known, the normalised Amari distance was measured, where 0 is perfect separation:

Batch outperforming online is expected, since it sees the whole recording repeatedly. ISS and IP landing on identical values is the signature of two methods minimising the same cost.

End-to-end reproduction. The repository was cloned into a clean virtual environment with current library versions and every script rerun, to confirm the results do not depend on the development machine's state.

6 Defects found, and what each cost

Each of the following changed reported numbers. They are recorded because each is easy to reintroduce.

1. Covariance recursion anchored to the wrong value. The recursion was applied to the running iterate rather than the previous frame, giving an effective forgetting factor of $\alpha^{N_{\text{iter}}}$ and injecting the frame's $\boldsymbol{x}\boldsymbol{x}^{\mathsf{H}}$ once per sweep instead of once per frame. Cost: about 1.5 dB of SDR. Symptom: none, it simply separated worse.

2. Signal-to-noise ratio off by 12 dB. The noise level was set with `10 ** (-snr / 10)` used as an *amplitude* scale, mixing the power and amplitude conventions. A nominal 15 dB SNR was in fact producing 27 dB. Every score measured before this fix was taken in a far cleaner room than claimed. The corrected form derives the noise from the measured signal level using `10 ** (-snr / 20)`, verified at 15.04 dB. This error was inherited from the upstream reference code and is still present there.

3. Conjugate on the wrong side of the Hermitian form. Discussed in Section 4.1. Produces plausible real numbers, so it must be caught numerically.

4. `einsum` versus BLAS. Discussed in Section 4.1. Before correction, the runtime comparison measured implementation style and inverted the conclusion above eight channels.

5. A 10 MB temporary in batch ISS. The natural way to write the numerator of (10),

```
v_num = (Y * r_inv[None, :, :]) @ np.conj(Y[:, s, :, None])
```

materialises a full (F, K, N) temporary on *every pivot*, about 10 MB at twelve channels. Batch ISS has low arithmetic intensity and is bandwidth-bound, so the memory traffic dominates. Measured 2.5× slower than the fused alternative,

```
v_num = np.einsum("fmt,mt,ft->fm", Y, r_inv, np.conj(Y[:, s, :]), optimize=True
    )
```

With the naive form the batch curve *fell* to 0.71× at sixteen channels, inverting the paper's result entirely. Note the irony against point 4: here `einsum` is faster, because avoiding the temporary matters more than reaching BLAS.

6. Benchmark workload too small to time. At low channel counts a batch sweep completes in a few milliseconds. With an insufficient workload the benchmark produced a spurious 3.03× spike at sixteen channels that vanished once the number of sweeps and trials was raised.

7 Behavioural findings

7.1 Scale drift during silence

During silence nothing is added to $\boldsymbol{U}$, so it decays geometrically as α^t. The self-normalisation term $1 - (\boldsymbol{w}_k^{\mathsf{H}} \boldsymbol{U}_k \boldsymbol{w}_k)^{-1/2}$ responds by inflating $\boldsymbol{w}_k$ to keep the quadratic form near unity, so the demixing matrix grows roughly as $\alpha^{-t/2}$. Measured on a synthetic mixture with a silent gap inserted:

Silent frames	$\boldsymbol{W}$ scale	Amari, first 30 frames after	Amari, 300 frames later
0	4.85	0.0146	0.0140
50	7.10	0.0241	0.0106
300	29.67	0.1203	0.0213
1000	96.65	0.1604	0.0142

Separation immediately after a long pause degrades by an order of magnitude, though it recovers within a few hundred frames. Nothing diverges: the ϵ floor in the denominator is load-bearing here, not cosmetic. Freezing the covariance update on silent frames would prevent the decay that drives this, and the per-source r_{kt} already provides a natural activity signal. This is the same mechanism as the paper's "flexible" update, reached from a different direction.

7.2 Projection back over long recordings

Because the demixing matrix keeps adapting, the output scale drifts, and a single coefficient fitted across a long file suits neither end. On two-minute material:

Projection back fitted	SDR (dB)	SIR (dB)
Once over 120 s	−3.45 / −2.13	16.01 / 22.09
Every 10 s	0.23 / 1.02	14.64 / 17.54
Every 2 s	2.22 / 2.21	13.34 / 13.40

Segmenting recovers about 5.7 dB of SDR, trading some SIR. Note that this makes the reported figure slightly optimistic relative to a true streaming system, which could not inspect a whole segment before choosing its scale.

7.3 Block size

Shorter blocks reduce latency but weaken the narrowband approximation of Section 1, since the window must be long relative to the room impulse response. At 15 dB SNR with $T_{60} \approx 300$ ms:

SIR barely moves while SDR varies by 2.5 dB. The short window separates about as well; it *reconstructs* worse, which is the time-domain aliasing that appears once the effective filter no longer fits inside the window.

Block	Hop	Window	Blocks	SDR (dB)	SIR (dB)
1024	512	64 ms	551	0.42	15.36
2048	1024	128 ms	276	2.27	15.24
4096	2048	256 ms	138	2.98	14.95

8 Complexity, and why batch and online differ

8.1 The offline argument

Per iteration and per frequency bin, IP builds an $M \times M$ covariance, $O(M^2 N)$, and solves, $O(M^3)$, repeated for M rows:

$$\mathcal{C}_{\text{IP}} = O\big(FM^3 \max(M, N)\big)\,. \tag{17}$$

ISS computes (10) for all m, k, each requiring $O(N)$ operations, and never forms a matrix:

$$\mathcal{C}_{\text{ISS}} = O\big(FM^2 N\big)\,. \tag{18}$$

The ratio grows with M, hence the widening gap offline.

8.2 Why it does not transfer

Online, (10) is unavailable. Those expressions are sums over N frames of already-separated outputs; with a single frame there is nothing to sum, and rank-1 statistics from one frame are useless. The recursion (12) therefore keeps $M \times M$ covariances in the *input* domain, and recovering the same quantities requires $\boldsymbol{U}_m \boldsymbol{w}_k$ for every m at every pivot: K matrix-vector products of $O(M^2)$, so $O(M^3)$ per pivot and $O(M^4)$ per sweep. IP is likewise $O(M^4)$. Both land at the same order, and only the constant differs.

Equivalently: batch ISS avoids matrices by working in the output domain, but online separation of the incoming frame requires $\boldsymbol{W}_{ft}$ explicitly, and the output-domain formulation never forms it.

Note that the original paper remarks that with $N = 1$ ISS is merely quadratic in M while IP can be brought to cubic via Sherman-Morrison. That analysis assumes working directly from the current frame's outputs. The recursive algorithm considered here does not, so the two statements describe different computations.

8.3 Measurements

Synthetic data, $F = 129$, $N = 400$, best of five trials, batch running 40 sweeps and online 2 per frame:

Channels	Batch ISS speedup	Online ISS speedup
2	0.83×	1.34×
4	0.82×	1.28×
6	1.18×	1.24×
8	1.12×	1.19×
10	1.29×	1.14×
12	1.29×	1.07×
14	3.11×	1.01×
16	2.68×	0.85×

The curves move in opposite directions. At two channels ISS is offline *slower*, since a 2×2 solve is trivial and ISS's overhead dominates.

For comparison, Figure 3 of the paper reports roughly 1.4× at twelve channels rising to about 1.8× at seventeen, from a multicore C++ implementation. Agreement at twelve channels is close. The shape reproduces; individual points at fourteen and sixteen are noisy in this measurement, batch ISS having measured marginally faster at sixteen than at fourteen, which is not physical. The paper likewise notes its own measured gap falls short of the complexity prediction, attributing this to a larger hidden constant in ISS and to memory access patterns.

9 Making the comparison fair

A timing comparison is only meaningful if the two modules differ solely in the update rule. Four changes were made to enforce this, and one proposed change was rejected on evidence.

Scale-matched prior. Both modules initialised their covariance to $0.01\,\boldsymbol{I}$, which is arbitrary relative to the signal level. The input power alone is also the wrong scale: the recursion accumulates $\varphi(r)\,\boldsymbol{x}\boldsymbol{x}^{\mathsf{H}}$, and φ carries units of inverse amplitude, so the covariance settles at power over amplitude rather than at power. Measured on a test signal, the steady-state mean diagonal was 0.265 while the mean input power was 18.04, an overshoot of 68×. Both modules now use

$$u_0 = \begin{cases} \bar{p}/\left(2\sqrt{\bar{e}}\right) & \text{Laplace,} \\ F\bar{p}/\bar{e} & \text{Gauss,} \end{cases} \tag{19}$$

with $\bar{p}$ the mean per-bin power and $\bar{e}$ the mean across-frequency energy over the first fifty frames. This lands within 1% of the settled value. The consequence is that separation quality becomes invariant to input gain: the Amari distance measured 0.0090 at every gain from 10^{-3} to 10^{3}, where previously the arbitrary prior made behaviour gain-dependent.

Two-pass sweep. Previously each source refreshed its own r_k and $\boldsymbol{U}_k$ as its turn arrived, so source m's covariance was current if m had already been visited in that sweep and stale otherwise, making results depend on source indexing. Both modules now refresh all statistics in a first pass, then sweep the pivots in a second. This matches the batch Algorithm 1, which computes the activations once per iteration and notes that this suffices. Agreement with the brute-force reimplementation improved from 10^{-14} to 10^{-15}.

Pass one is byte-identical between the modules. Verified textually, not by eye: the two blocks differ only in the covariance variable's name.

A NumPy 2.0 incompatibility in the IP solve. The right-hand side was passed with shape (F, M), which NumPy 1.x interprets as a stack of vectors. NumPy 2.0 changed that rule and raises `ValueError` unless $F = M$. Confirmed by running under both versions. The trailing axis is now explicit, and both modules were verified to run under NumPy 2.0.2 as well as 1.23.

Regularising the right matrix. The IP solve was loaded as $\boldsymbol{W}\boldsymbol{V}_k + \epsilon\boldsymbol{I}$. That product is not Hermitian, so adding to it is not regularisation in the usual sense. The loading now applies to $\boldsymbol{V}_k$ itself, which is Hermitian positive semi-definite, and the same loaded matrix is reused for the normalisation so the two remain consistent. The constant is also now relative, $10^{-6}u_0$, rather than absolute: a fixed 10^{-10} does not track the input gain and would not prevent a `LinAlgError` on a quiet stretch, which in a streaming run is fatal, whereas ISS merely degrades through its denominator floor.

Two smaller items. The per-source weight φ is already vectorised, so the list comprehension around it was removed in both modules, keeping pass one byte-identical. And the transposed view of $\boldsymbol{W}$ used in the ISS numerator is now copied: it was safe only because the numerator happened to be materialised before $\boldsymbol{W}$ was modified three lines later, which is an accident of statement order rather than something enforced.

Rejected: a warm-up gate on the ISS pivot rescaling. It was proposed to disable ISS's self-normalisation for the first $3/(1-\alpha)$ frames. This was tested rather than assumed:

	Amari, frames 0–200	frames 200–600	steady state
ISS without gate	0.3959	0.0465	0.0085
ISS with gate	0.4259	0.0476	0.0085
IP (no counterpart exists)	0.4005	0.0473	0.0085

The gate makes the early window slightly *worse* and changes nothing afterwards. More importantly, IP's normalisation is not optional, so no equivalent gate exists for it: adding one to ISS alone would introduce an asymmetry rather than remove one. The measured transient is already symmetric, 0.3959 against 0.4005, so there is nothing to correct. With the power-matched prior in place the gate addresses a problem that no longer exists.

Rejected: collapsing the rank-1 term to avoid materialising $\boldsymbol{U}$. Since $\boldsymbol{x}\boldsymbol{x}^{\mathsf{H}}\overline{\boldsymbol{w}_k} = \boldsymbol{x}\,\overline{y_k}$, the new term of the recursion collapses and $\boldsymbol{U}$ need never be formed, carrying only $\boldsymbol{U}_{t-1}$. The identity is exact, verified to 2×10^{-15}, and the restructured code passed the brute-force check unchanged. It was nonetheless reverted, because it is slower precisely where it was expected to help. Interleaved best-of-nine timings, ISS only:

Channels	Materialised $\boldsymbol{U}$ (ms/frame)	Collapsed (ms/frame)	Gain
2	0.39	0.45	0.85×
3	0.84	0.96	0.88×
4	1.31	1.46	0.89×
6	3.70	3.94	0.94×
8	8.91	8.99	0.99×
12	32.84	31.54	1.04×

The premise was that allocation overhead dominates at small M. It does not. Both forms perform the same K^2F matrix-vector products; the collapse trades one (K, F, M, M) allocation per sweep for K additional elementwise operations per sweep, and with $K = M$ those are the same order. What remains is extra interpreter and temporary-allocation overhead per pivot, which costs more than the saved array precisely at the small channel counts where ISS holds its advantage. It only breaks even at eight channels and is marginally ahead at twelve.

Applied: distributive diagonal loading in IP. The loaded matrix is no longer materialised, using $\boldsymbol{W}(\boldsymbol{V}_k + \rho\boldsymbol{I}) = \boldsymbol{W}\boldsymbol{V}_k + \rho\boldsymbol{W}$ and likewise for the normalisation. It was *not* folded into pass one, which would be simpler: the snapshot $\boldsymbol{V}_{t-1}$ would then inherit the loading each frame and compound it to $\rho/(1-\alpha)$, a 25× inflation at $\alpha = 0.96$, confirmed by simulating the recursion.

Applied: the loading scale follows a supplied prior. When the caller passes $\boldsymbol{V}_0$ it may sit at a different scale than the input-derived value, so ρ is now measured from the supplied matrix's mean diagonal rather than assumed.

On the remaining asymmetry. IP carries diagonal loading and ISS does not. This is deliberate and is not an unfair advantage handed to ISS: a singular solve fails hard, whereas ISS degrades gracefully through the floor on its scalar denominator. Removing the loading would not equalise the comparison, it would make IP crash on quiet input. The module header now says this rather than claiming outright parity.

What fairness does not buy. These changes make the *implementations* comparable. They do not change the fact, established in Section 8, that both online rules are $O(M^4)$ per sweep, so any measured speedup reflects BLAS-3 matrix multiplication outperforming LAPACK's batched solve rather than the inverse-free advantage the ISS literature reports. Captioning such a plot as demonstrating that advantage would be wrong. The honest framing is the one used throughout this report: the online reformulation narrows the gap between IP and ISS relative to the batch case.

10 Results on speech

Two-speaker reverberant mixtures, 15 dB SNR, 1024-sample blocks.

Material	Algorithm	SDR (dB)	SIR (dB)	ms/block	Realtime factor
4.4 s clips	ISS	0.59 / 0.25	12.94 / 17.79	0.70	0.022
4.4 s clips	IP	0.59 / 0.25	13.09 / 17.77	1.13	0.035
120 s recordings	ISS	2.22 / 2.21	13.34 / 13.40	0.75	0.023
120 s recordings	IP	2.23 / 2.25	13.34 / 13.77	1.17	0.037

Three observations. The two algorithms agree to within 0.04 dB throughout, as they must. Per-block cost is flat between 551 and 3767 blocks, confirming the method is genuinely online: cost does not grow with how much audio has already been processed. And a realtime factor near 0.02 leaves roughly 45× headroom, though in pure Python on a desktop CPU with only two channels, which is the easy case.

11 Limitations

- **The flexible update is not implemented.** Every source is updated on every sweep. Restricting updates to moving sources is where the larger online saving is expected, and Section 7.1 suggests the same mechanism would address the silence problem.
- **Projection back is not causal.** The batch form reads future frames. Segmenting mitigates but does not eliminate this; a true streaming system needs a recursive scale estimate.
- **Determined case only.** Microphones must equal sources.
- **Pure Python.** A compiled or embedded target would shift every constant reported here, and the realtime headroom in particular should not be extrapolated to a phone or DSP.

12 Repository contents

Source: `https://github.com/Joels1994/online-auxiva-iss`

Attribution. `projection_back.py`, `metrics.py` and the batch implementations derive from `piva` by Robin Scheibler (GPL-3.0). Speech is from the CMU ARCTIC corpus. The batch algorithms are due to Ono and to Scheibler and Ono; the online ISS formulation to Nakashima and Ono.

File	Purpose
`online_iva/auxiva_iss_online.py`	Online ISS
`online_iva/auxiva_ip_online.py`	Online IP baseline
`online_iva/batch_reference.py`	Batch ISS and IP
`online_iva/projection_back.py`	Scale resolution
`online_iva/metrics.py`	SDR/SIR evaluation
`verify_online.py`	Brute-force cross-check and Amari test
`demo_audio.py`	Speech demo, `--long` for 2-minute material
`demo_blocksize.py`	Block-size study
`benchmark_runtime.py`	Online ISS versus IP scaling
`benchmark_batch_vs_online.py`	All four, 2 to 16 channels